Final project
Building and Improving a
Local Retrieval-Augmented Generation System

Introduction and context

Large Language Models and their limitations

Large Language Models (LLMs) have transformed the field of Artificial Intelligence (AI) by demonstrating remarkable capabilities in natural language understanding, question answering, text generation, summarization, and reasoning. Modern LLMs are trained on massive collections of text and can generate fluent and coherent responses across a wide range of topics. Despite their impressive performance, LLMs have several important limitations. Their knowledge is limited to the data available during training and may become outdated over time. Furthermore, LLMs can generate factually incorrect information, a phenomenon commonly known as hallucination. In addition, they often lack access to organization-specific information such as internal reports, policies, manuals, research papers, or proprietary datasets. These limitations reduce their reliability in applications that require accurate and up-to-date information.

Why Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG) was developed to address many of the limitations of standalone LLMs. Instead of relying entirely on the model's internal knowledge, a RAG system retrieves relevant information from an external document collection and provides that information as context to the language model before generating a response. By incorporating retrieved documents into the generation process, RAG systems can produce answers that are grounded in specific sources of information. This approach improves factual accuracy, reduces hallucinations, and enables the system to answer questions about information that was never included in the model's training data. As a result, RAG has become one of the most widely adopted architectures for enterprise AI assistants, document question answering systems, and knowledge management applications.

Why local RAG systems matter?

Many modern AI applications rely on cloud-based language models and external APIs. While these solutions are convenient, they may introduce concerns related to privacy, cost, internet dependency, and limited system transparency. Local RAG systems address these challenges by running all components directly on a local machine. Documents remain stored on the local machine, reducing privacy risks and eliminating the need to upload sensitive information to external services. Local deployment also avoids recurring API costs and enables the system to operate even without an internet connection. Furthermore, local systems provide students and researchers with greater visibility into the entire RAG pipeline, making them valuable platforms for experimentation and learning. The ability to build and deploy efficient local AI systems has become increasingly important in domains such as education, healthcare, finance, government, and enterprise environments where data privacy and system control are critical requirements.

Project overview

In this project, students will build, analyze, and improve a complete local RAG system that runs entirely on a personal computer using open-source tools. The provided baseline implementation utilizes local embedding models, a vector database, and a local Large Language Model served through Ollama.

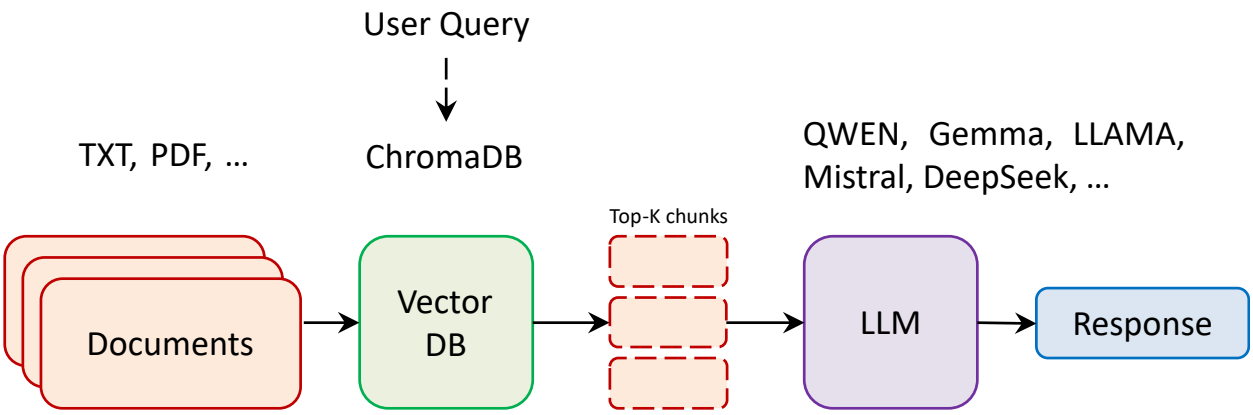
Students will study the architecture of the baseline system, evaluate its strengths and weaknesses, and develop techniques to improve retrieval quality, answer quality, efficiency, and usability. The project combines concepts from information retrieval, natural language processing, machine learning, and Generative AI, providing hands-on experience with technologies that are increasingly used in real-world AI applications.

Upon successful completion of this project, students should be able to:

1. Explain the principles of RAG, describe the role of its main components, and discuss how integrating external knowledge improves the reliability and usefulness of AI generated responses.
2. Design, implement, and operate a complete local RAG pipeline, including document processing, embedding generation, vector storage, information retrieval, prompt construction, and answer generation.
3. Evaluate the performance of a RAG system through systematic experiments and analyze the impact of key design choices such as chunking strategies, embedding models, retrieval settings, and language models.
4. Identify limitations of a baseline RAG implementation and develop, implement, and compare techniques that improve retrieval quality, answer quality, efficiency, or usability.
5. Communicate technical findings and experimental results effectively through source code, documentation, written reports, visualizations, and oral presentations.

Components of a RAG pipeline

A RAG system combines information retrieval with a LLM to generate answers grounded in external knowledge. Instead of relying solely on the knowledge learned during training, the system retrieves relevant information from a document collection and uses it to answer user questions. A typical RAG pipeline consists of the following components:



- Documents: Documents serve as the knowledge source of the system. They may consist of company policies, technical manuals, research papers, textbooks, product documentation, web pages, customer support articles, or other collections of information. The quality and coverage of these documents directly affect the quality of the answers produced by the RAG system.
- Vector Database (Vector DB): Before documents can be searched efficiently, they are divided into smaller chunks and converted into vector embeddings. These embeddings are stored in a vector database, which is optimized for semantic similarity search. Popular vector databases include ChromaDB, FAISS, Qdrant, and Milvus. This component allows the system to find information based on meaning rather than exact keyword matches.
- Top-K Retrieved Chunks: When a user submits a question, the system searches the vector database and retrieves the K most relevant document chunks. These chunks represent the pieces of information that are most likely to contain the answer.
- Large Language Model: The retrieved chunks and the user's question are combined into a prompt and sent to a LLM. Models such as Qwen 3, Gemma 3, Llama 3.1, and Mistral can analyze the provided context, identify relevant information, and generate a coherent response. The quality of the model influences the accuracy, completeness, and fluency of the final answer.
- Response: The final output of the RAG system is a natural language response generated by the LLM using the retrieved context. A well-designed RAG system produces answers that are accurate, relevant to the user's question, and grounded in the supporting documents. Some systems may also provide citations or references to indicate where the information originated.

Run the sample code

- ***Step 1: Install Ollama***

Ollama allows you to run LLMs locally on your computer without relying on cloud services. It manages model downloads and provides an API that the Python program can access.

Install the Ollama server:

```
https://ollama.com/download
```

Start the Ollama server:

```
ollama serve
```

The default server address is:

```
http://127.0.0.1:11434
```

- ***Step 2: Download required embedding and LLM models***

The RAG system requires two models:

- An embedding model that converts text into numerical vectors for document retrieval.
- A language model that generates answers.

Download the embedding model:

```
ollama pull nomic-embed-text
```

Download the language model:

```
ollama pull qwen2.5:0.5b
```

Verify that the models are installed:

```
ollama list
```

- ***Step 3: Create a Python environment***

A Python environment keeps project dependencies separate from other Python projects on your computer. This avoids package conflicts and makes the project easier to manage.

Install Anaconda:

```
https://www.anaconda.com/docs/getting-started/anaconda/install/overview
```

Create a new Conda environment:

```
conda create -n rag python=3.10
```

Activate the environment:

```
conda activate rag
```

Install the requirements of the sample code:

```
pip install -r requirements.txt
```

- ***Step 4: Run the code***

The Python script needs a folder containing text documents that will serve as the knowledge base for the RAG system. Currently, the sample code only supports *.txt files.

The sample code has the following structure:

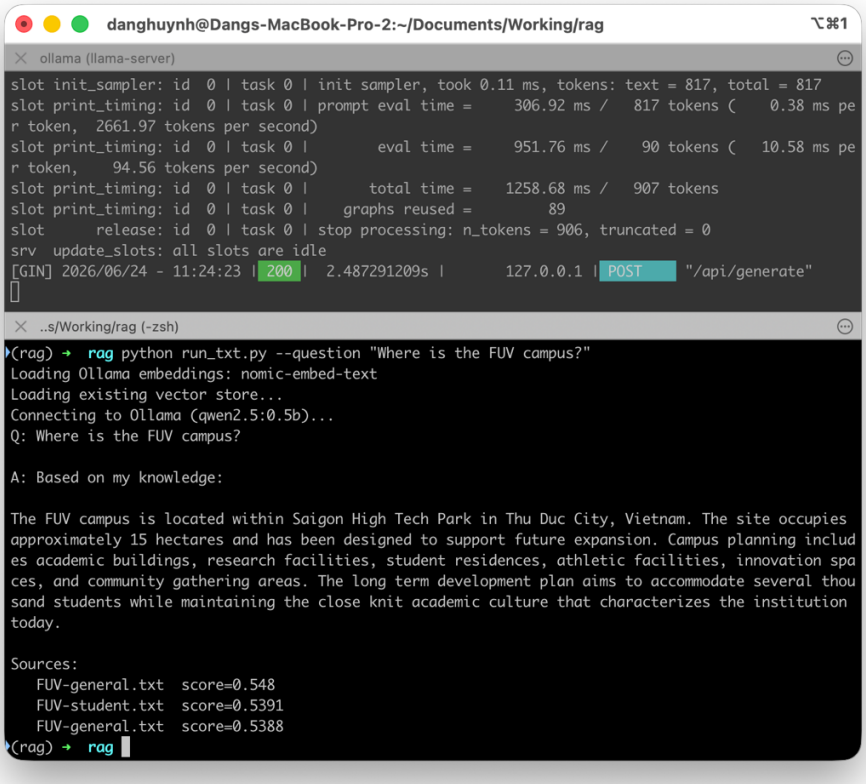
```
project/
├── run_txt.py
└── data_txt/
    ├── vector_store.txt
    ├── document1.txt
    ├── document2.txt
    └── document3.txt
```

Run with the default question:

```
python run_txt.py
```

Run with your own question:

```
python run_txt.py --question “Where is the FUV campus”
```



Note on Operating System: The sample code has been successfully tested on macOS (Tahoe 26.5.2) and Linux (Ubuntu 26.04 LTS). It should also work on other versions of macOS and Linux with little or no modification. If you are using Windows, it is recommended to run the code on a Linux environment, either through a virtual machine or by installing Linux on your local machine. Becoming familiar with Linux is an important skill for AI development and research, as many AI tools and frameworks are designed primarily for open source operating systems. If you choose

to continue using Windows, you will need to determine how to adapt and run the sample code on your own system. The following resources may be helpful:

- Install Ollama on Windows: https://ollama.com/download/windows
- Install Anaconda on Windows: https://www.anaconda.com/docs/getting-started/installation
- Install Docker Container on Windows: https://docs.docker.com/desktop/setup/install/windows-install/

Project requirements

This project comprises seven (07) main tasks. The following table provides a detailed overview of the project requirements.

# Task	Name	Requirements
Task 1 (5%)	PDF file support	The baseline implementation only supports text (*.txt) files as document sources. Extend the code to support PDF documents. The system should automatically extract text from PDF files, split the extracted content into chunks, generate embeddings, and index the resulting chunks in the vector database. Students may select any PDF document of their choice for testing purposes.
Task 2 (15%)	Major handbook chatbot	Select ONE major handbook (PDF files are provided in the sample code folder) and develop a RAG-based chatbot that can answer questions about the handbook. The chatbot should be able to respond to topics such as program requirements, course offerings, learning outcomes, and academic policies. The responses are not required to be completely correct, as the performance of a RAG system depends on many factors, including the language model, embedding model, chunking strategy, retrieval settings, and prompt design. Document examples of questions that are answered correctly, as well as questions that are answered incorrectly, incompletely, or ambiguously, and discuss possible reasons for these outcomes.
Task 3 (20%)	Docker container deployment	After developing the RAG-based chatbot that can successfully retrieve information from the selected handbook, deploy the application using Docker containers to create a portable and reproducible environment. The deployment should include a Dockerfile, container configuration, and clear instructions for building and running the system.
Task 4 (20%)	UI/UX deployment	Build a user friendly web interface that allows users to interact with the chatbot through a browser. The interface should support question input, answer display, and basic conversation history.

Task 5 (15%)	Report	Submit a final report of at least 1,500 words describing the design, implementation, and evaluation of your complete RAG system. The report should discuss observations made during development, including retrieval performance, model behavior, prompt design, limitations, failure cases, lessons learned, and possible future improvements.
Task 6 (20%)	Video demo	Students will submit a video demo of their RAG system. The video should clearly show how each project task (from task 1 to task 4) is completed. For each task, students should demonstrate the implementation process, explain its purpose, and present the resulting output. The video should provide sufficient evidence that all project requirements have been successfully completed and are functioning as intended. There are four video demos required: • Video demo for task 1 (5%) • Video demo for task 2 (5%) • Video demo for task 3 (5%) • Video demo for task 4 (5%)
Task 7 (5%)	Github repository and README	Students will submit a GitHub repository containing all source code, configuration files, and project assets required to reproduce the RAG system. The repository should include a well organized README file with installation instructions, dependency requirements, usage examples and Docker deployment steps.
100%	Total	

Project Instructions

Note: These are example approaches rather than strict requirements. Students have the flexibility to explore and implement their own solutions.

- ***Task 1: PDF file suport***

The sample code only supports text files as document sources. We will have to extend the system to support PDF documents.

Suggestions:

- Install a Python library for PDF processing, such as PyPDF.
- Replace the text file loader with a PDF loader.
- Extract text from PDF files.
- Split the extracted text into chunks.
- Generate embeddings and store them in the vector database.
- Verify that the chatbot can answer questions using information from PDF documents.

Deliverables:

- Modified source code
- Demonstration of successful PDF retrieval

- ***Taks 2: Major handbook chatbot***

Select one major handbook provided in the project folder and use it as the knowledge source for a RAG based chatbot.

Suggestions:

- Select one of the major handbook PDF files (provided in the sample code folder).
- Build a vector database from the handbook.
- Design a set of test questions covering different sections of the handbook.
- Evaluate chatbot responses.
- Record examples of correct answers, incorrect answers, incomplete answers, and ambiguous answers.
- Compare the performance of different language models if possible.

Deliverables:

- Working chatbot
- Test questions and results
- Analysis of chatbot performance

- ***Task 3. Docker container deployment***

Package the chatbot into a Docker container to create a portable and reproducible deployment environment.

Suggestions:

- Install Docker (https://www.docker.com/).
- Create a Dockerfile for the application.
- Install all required dependencies inside the container.
- Configure volume mounting for documents and vector databases.
- Build the Docker image.
- Run and test the containerized chatbot.
- Document all deployment steps.

Deliverables:

- Dockerfile
- Container configuration files
- Deployment instructions

- ***Task 4. UI/UX deployment***

Develop a web based interface that allows users to interact with the chatbot through a browser.

Suggesttions:

- Select a web framework such as Streamlit, Gradio, Flask, or FastAPI.
- Create an interface for entering questions.
- Display chatbot responses in a readable format.
- Implement basic conversation history.
- Connect the interface to the RAG backend.
- Test the complete system using multiple questions.

Deliverables:

- Web interface
- Backend integration
- Screenshots of the interface

- ***Task 5. Report***

Submit a final report of at least 1,500 words describing the design, implementation, and evaluation of the complete RAG system.

Suggestions:

- System architecture
- Chatbot evaluation
- Docker deployment
- User interface design
- Observations and lessons learned
- Limitations and future improvements

Deliverables:

- PDF report

- ***Task 6. Video demo***

Create a video demonstration of the completed project.

Suggestions:

- Task 1: demonstrate PDF processing.
- Task 2: demonstrate the handbook chatbot.
- Task 3: demonstrate Docker deployment.
- Task 4: demonstrate the web interface.
- Show example questions and answers.

Deliverables:

- Video demo for task 1.
- Video demo for task 2.
- Video demo for task 3.
- Video demo for task 4.

- ***Task 7. GitHub repository and README***

Submit a GitHub repository containing all source code, configuration files, and project assets required to reproduce the project.

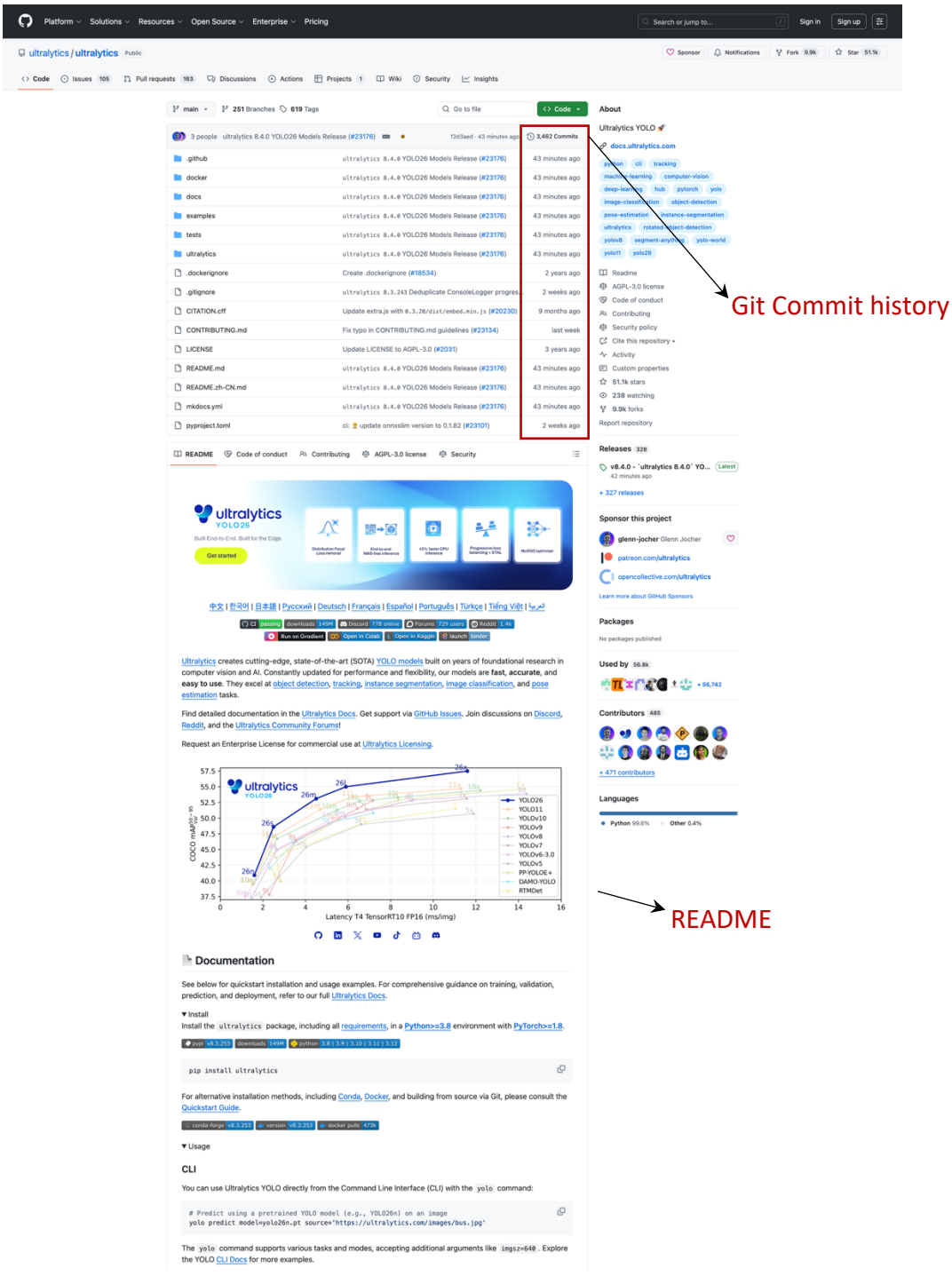
Suggestions for README:

- Installation instructions.
- Required dependencies.
- Docker deployment.
- Example usage.
- Known limitations.

Deliverables:

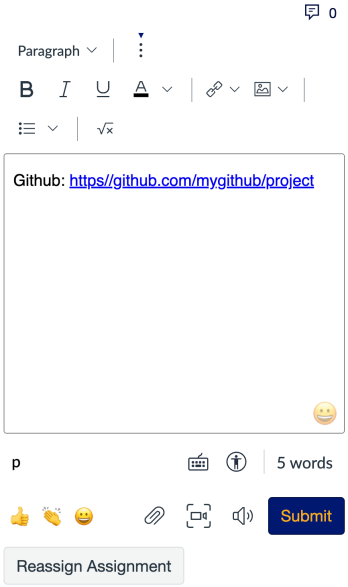
- Github repository
- README

Here follows an example of a GitHub repository with a well organized commit history and a comprehensive README.



Example of a GitHub Repository with a proper README
(https://github.com/ultralytics/ultralytics)

Please make sure to include your GitHub link in the comments section on Canvas:



Submission

The submission folder should contain the following sub-folders:

- `./source_code/`: contains all source code and project files.
- `./docker/`: contains the Dockerfile and related deployment files.
- `./report/`: contains the final report in PDF format.
- `./demo_video/`: contains the video demos.

The submission folder is named ADL-<StudentID>-project (e.g., ADL-2012345-project) and then compressed with the same name.

Note: If your video demos exceed the Canvas upload size limit, please host them externally (e.g., Google Drive or OneDrive) and include the access link in your report or Canvas comments.

Evaluation

Project evaluation will be based on successful task completion. Here's a breakdown of the grading criteria:

Task	Description	Grade	
Task 1	PDF file support	5%	5%
Task 2	Major handbook chatbot	15%	15%
Task 3	Docker container deployment	20%	20%
Task 4	UI/UX deployment	20%	20%
Task 5	Report	15%	15%
Task 6	Video demo for task 1	5%	20%
	Video demo for task 2	5%	
	Video demo for task 3	5%	
	Video demo for task 4	5%	
Task 7	Github and README	5%	5%
Total		**100%**	

Frequently Asked Questions (FAQs)

1. *Can I use a different embedding model or language model?* Yes. You may use any embedding model or language model that can run on your computer. Different models may produce different results, and these observations should be discussed in the report.
2. *Can I use a different vector database?* Yes. Chroma is used in the sample code, but you may use another vector database if you prefer.
3. *Can I use a PDF document other than the provided major handbooks for Task 1?* Yes. For Task 1, any PDF document may be used for testing purposes.
4. *Must Task 2 use one of the provided major handbooks?* Yes. Task 2 requires selecting one major handbook provided in the project folder.
5. *Can a RAG system retrieve text that appears inside images in PDF files?* By default, no. Most RAG systems extract only the selectable text from PDF files. If a PDF contains images with embedded text, such as scanned pages, screenshots, or diagrams, that text cannot be retrieved unless an Optical Character Recognition (OCR) step is applied during document processing. After OCR converts the text in the images into machine readable text, it can be indexed and retrieved by the RAG system.
6. *Why does the chatbot sometimes generate incorrect answers even when the information exists in the documents?* RAG systems can still produce incorrect, incomplete, or hallucinated answers. This is a well known limitation of LLMs. Even when relevant documents are retrieved, the model may misinterpret the retrieved content, combine information incorrectly, or generate unsupported statements. Factors such as retrieval quality, chunking strategy, prompt design, and model size can all affect answer accuracy. Students are expected to identify and discuss these cases in their report rather than assuming the system will always produce correct answers.
7. *Should I test different LLM architectures and model sizes to evaluate their performance?* RAG systems can still produce incorrect, incomplete, or hallucinated answers. This is a well known limitation of LLMs. Even when relevant documents are retrieved, the model may misinterpret the retrieved content, combine information incorrectly, or generate unsupported statements. Factors such as retrieval quality, chunking strategy, prompt design, and model size can all affect answer accuracy. Students are expected to identify and discuss these cases in their report rather than assuming the system will always produce correct answers. Please note that changing the language model may also require selecting a different embedding model to achieve good retrieval performance. Students are encouraged to experiment with alternative models such as `qwen2.5:1.5b` and `qwen2.5:3b`, subject to the computational resources available on their local machine.
8. *Must I complete the tasks in the specified order?* Yes, it is highly recommended. The tasks are designed to build on one another, so following the suggested order will make the project easier. However, you may organize your development process differently.
9. *Can I work in a group?* No. This is an individual project. Working independently ensures that each student gains hands-on experience building and managing the entire RAG pipeline.
10. *What should be shown in the video demos?* The videos should clearly demonstrate how Tasks 1 through 4 were completed and show the outputs produced by each task.

11. *Can I implement additional features beyond the required tasks?* Yes, you are welcome to include additional features in your report and demo videos. However, please note that these extra additions will not be graded.

Deadline

Please visit Canvas for details.